PUC Minas
Instituto de Ciências Exatas e Informática

Pontifícia Universidade Católica de Minas Gerais
Bacharelado em Engenharia de Software
Laboratório de Computação II
Prof.: Henrique Batista da Silva
Lista de Exercícios 4 – 1,0 ponto. Entrega: 19/03/18 (SGA)

Orientação à Objetos em Java

Orientações:

- Os exercícios devem ser feitos em **linguagem Java**.
- Cada exercício deve ser feito em um arquivo .java (extensão dos arquivos da linguagem Java). Você deve entregar apenas um único arquivo compactado (.zip) no SGA contendo todos os exercícios resolvidos.
- Exercícios copiados receberão nota zero.
- Não deixe a lista para a última hora. Comece o quanto antes. Assim haverá tempo para esclarecer dúvidas com o professor e na monitoria.

1- Considere a classe **Vetor**, cuja assinatura está definida abaixo:

```
public class Vetor {

        public static final int TAMANHO = 10;

        private int v [] = new int[TAMANHO];  // Armazena os elementos
        private int numElementos;   // Informação sobre o número de elementos inseridos

        public Vetor() { }

        public int obtemTamanho() { return 0; }

        public void insereNoFinal(int novoElemento) { }

        public int posicaoDe (int elemento) { return 0; }

        public void alteraEm (int pos, int novoValor) { }

        public int elementoDe (int pos) { return 0; }

        public int elementoEm (int pos) { return 0; }

        public void reverte() { }

        public void imprime() { }

}
```

Implemente todos os métodos da classe **Vetor** acima, obedecendo às seguintes descrições dos métodos:

a. Vetor(): Construtor que inicializa o atributo numElementos com zero e zera todas as posições do vetor;
b. obtemTamanho(): Retorna o número de elementos armazenados no vetor;
c. insereNoFinal(novoElemento): Insere novoElemento na primeira posição vazia do vetor;

d. posicaoDe(elemento): Retorna a posição da primeira ocorrência de elemento no vetor ou o valor -1, caso o elemento não seja encontrado.
e. alteraEm(pos, novoValor): Atribui novoValor ao elemento da posição pos do vetor, caso esta seja uma posição válida, retorna -1, caso contrário.
f. elementoEm(pos): Retorna o valor armazenado na posição pos, caso esta seja uma posição válida, retorna -1, caso contrário.
g. imprime(): Imprime os elementos do vetor, separados por espaço.
h. reverte(): Reverte os elementos do vetor. Por exemplo, o vetor inicial {1,3,4,1,2}, após a chamada desta função, o vetor ficará {2,1,4,3,1,};

Após a implementação da classe Vetor, a função main, a seguir, deverá funcionar:

```
public class Program {

      public static void main(String[] args) {
            // TODO Auto-generated method stub

       Vetor V = new Vetor();

        V.insereNoFinal(10); V.insereNoFinal(8);
        V.insereNoFinal(16); V.insereNoFinal(7);
        V.insereNoFinal(5);  V.insereNoFinal(13);

        V.imprime();
        V.alteraEm(3,19); V.alteraEm(15,9);
        for (int i = 0; i < V.obtemTamanho(); i++)
             System.out.println("Elemento na posicao " + i + ": " + V.elementoEm(i));

        V.reverte();
        V.imprime();

       Vetor v = new Vetor();

      }
}
```

2- Nesta aula, você deverá alterar a classe Vetor (definida anteriormente) para:

a. Permitir a realocação do vetor ao se tentar inserir um elemento e não houver posições vazias. A cada realocação você deverá dobrar o tamanho do vetor. Inicie com 10 elementos.

b. Permitir a retirada de todas as ocorrências de um elemento. Ao remover um elemento, se este não for o último elemento, deve-se deslocar todos os demais para não deixar posições vazias no vetor. Se o número de elementos após a retirada ficar menor que a metade do tamanho do vetor, então você deverá realocar o vetor com a metade do número de elementos.

c. Modifique o programa principal (função main) para testar se novo programa.

3- Considere que você foi contratado para desenvolver um sistema para um Banco.

No sistema bancário precisamos armazenar as informações dos clientes, como número e nome. Crie uma classe para modelar os objetos que representarão os clientes.

```java
public class Cliente {

       private int Numero;
       private String Nome;

}
```

Crie um construtor para a classe Cliente

```java
public Cliente (String nome, int numero)
{
      this.Nome = nome;
      this.Numero = numero;
}
```

Crie também a classe conta com os seguintes atributos:

```java
public class Conta {

       private int Numero;
      private double Saldo;
      public Cliente Titular;
}
```

Crie também o construtor da classe conta

```java
public Conta (Cliente cliente, int numero)
{
        this.Titular = cliente;
        this.Numero = numero;
        this.Saldo = 0
}
```

Suponha que o banco precise realizar operações de saque, depósitos e transferências. No entanto, as operações de saque e transferência possuem algumas regras. Há uma taxa de 10% sobre o valor da operação para cada retirada (seja ela do tipo saque ou transferência).

Para a classe Conta inclua os métodos deposita, saque e transfere.

```java
public void deposita(double valor) { }

public double saque(double valor) { return 0.0; }

public double transfere (double valor, Conta destino) { return 0.0; }
```

Outra regra é que o banco não permite que o cliente tenha saldo negativo em sua conta. As operações de saque e transferência devem tratar para que o saldo não fique negativo antes de realizar a operação.

Implemente o programa conforme especificado abaixo:

a. Implemente as operações de saque, transferência e depósito de cada conta. Atente-se que para as operações de saque e transferência é necessário verificar se há saldo suficiente antes de realizar a operação.

b. Implemente uma operação para verificar o saldo do cliente (observe a visibilidade do atributo saldo da classe conta).

c. Implemente um programa principal para criar dois clientes e duas contas para cada um dos clientes e proceda com as operações de depósito para cada cliente, saque e transfira uma quantia de valor de um cliente para o outro. Seu programa deve imprimir na tela a ocorrência de cada operação (ex.: cliente A transferiu R$ 10,00 para o cliente B).

d. Ao final, seu programa deverá imprimir na tela o nome de cada cliente e o saldo de suas respectivas contas.

e. Crie testes unitários usando o jUnit para testar os métodos *deposita*, *saque* e *transfere* (opcional).